

Artificial Intelligence Laboratory

Lab Experiment 6 — The 8-Queens Problem

DFS · BFS · Best-First Search · A* Algorithm
6th Semester · B.Tech CSE · NIT Srinagar · Spring 2026

Student Name		Enrollment No.	
Batch		Date	

Instructions: Answer all 4 problems. For each problem: (a) write the algorithm / pseudocode, (b) write the complete Python solution, (c) record the program output by running your code, and (d) answer the analysis questions. All four problems use the **same 8-Queens puzzle** but with a different search strategy each time.

Quick Reference — 8-Queens as a Search Problem

Search cept	con-	8-Queens meaning	Implementation
State		Partial placement: queens in rows 0...k	tuple of column positions
Start state		Empty board	()
Goal state		8 queens placed, none at- tacking	(c0,c1,...,c7) valid
Action		Place next queen in a safe column	Append one column value
Constraint		No two queens attack each other	is_safe() check

Core safety check: Two queens at (r_1, c_1) and (r_2, c_2) attack on a diagonal when $|r_1 - r_2| = |c_1 - c_2|$. The row constraint is satisfied automatically by placing exactly one queen per row.

Algorithm	Data Structure	Finds all 92?	Uses heuristic?	Memory
DFS	Stack (recursion)	Yes	No	Low
BFS	Queue (deque)	Yes	No	High
Best-First	Min-Heap	No (first)	$h(n)$ only	Medium
A*	Min-Heap	No (first)	$g + h$	Medium

1

Scenario 1: Security Camera Placement in a Museum

A museum security team must install **8 security cameras** in an 8×8 grid of gallery rooms. Each camera covers its entire row, column, and both diagonals (exactly like a chess queen). Find a placement so that **no two cameras have overlapping lines of sight**.

Strategy: Depth First Search (DFS) with backtracking. Place cameras row by row, trying each column in order. When a conflict is found, backtrack and try the next column. Continue until all 92 valid arrangements are found.

DFS hint: The recursive call `dfs(state + (col,))` is the “go deeper” step. Simply returning from a call (when no column is safe) *is* the backtrack step — no extra code needed. Use a `solutions` list passed by reference to collect all 92 results.

Tasks:

1. Write the `is_safe(state, col)` function that checks the column and diagonal constraints. [3 Marks]
2. Write the recursive `dfs_queens(state, solutions)` function in pseudocode, then in Python. [6 Marks]
3. Write a `print_board(solution)` helper that displays the board as an 8×8 grid. [3 Marks]

4. Run your program and record: total solutions found, the first solution (1-indexed columns), and the board diagram. [4 Marks]

5. **Analysis:** Without the `is_safe()` check, the search space is 8^8 . With column-uniqueness enforced, it is $8!$. DFS with `is_safe()` explores far fewer nodes. Explain *why* backtracking prunes the search tree effectively. [4 Marks]

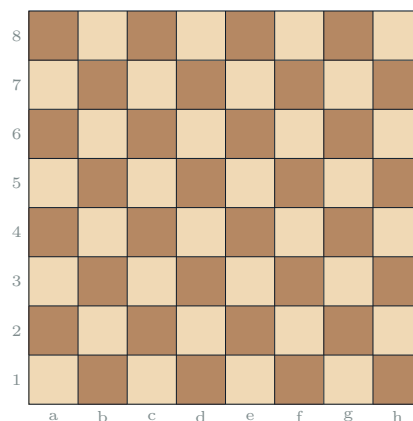
(a) Pseudocode for `is_safe` and `dfs_queens`:

(b) Python implementation:

(c) Complexity table (fill in):

Property	Value	Explanation
Time complexity (worst)		
Space complexity		
Nodes explored ($N = 8$)		
Solutions found		

(d) Program output — draw the first solution on the board below and fill in the columns:



Solution (1-indexed columns): _____

Total solutions found: _____

(e) Analysis — why does backtracking prune the search tree?

Scenario 2: Military Parade Ground Formation

A military commander must arrange **8 flag-bearers** on an 8×8 parade ground, one per row, so that no two flag-bearers share a column or diagonal. The commander wants to explore all formations **level by level** to understand how many valid partial formations exist at each depth.

Strategy: Breadth First Search (BFS). Use a deque as a queue. Level 0 holds the empty board; Level 1 holds all single-queen placements; ...; Level 8 holds complete valid solutions. Process states in FIFO order.

BFS hint: Import from collections import deque. Start with queue = deque([()]). At each step, popleft() a state, expand it by appending all safe next columns, and push them back with append(). Track nodes_explored to compare with DFS.

Tasks:

1. Write the `bfs_queens()` function. It should return all 92 solutions and the count of nodes explored. **[8 Marks]**
2. Run your program and record: nodes explored, first solution, last solution. **[4 Marks]**
3. Fill in the queue-size table below based on your program or analysis. **[3 Marks]**
4. **Analysis:** BFS and DFS both find 92 solutions, but BFS explores far fewer nodes yet uses much more memory. Explain this trade-off. Why is BFS generally *not* preferred for constraint satisfaction problems? **[5 Marks]**

(a) Python implementation of `bfs_queens()`:

(b) Queue size at each level (fill in from your run or analysis):

Level (queens placed)	Valid states in queue	Explanation
0	1	Empty board
1		
2		
3		
...	...	...
8	92	Complete solutions

(c) Program output:

Nodes explored: _____ Solutions found: _____

First BFS solution (1-indexed): _____

Last BFS solution (1-indexed): _____

(d) Analysis — BFS vs DFS trade-off for constraint satisfaction:

Scenario 3: Satellite Dishes on a Grid Rooftop

A telecommunications company must place **8 satellite dishes** on a rooftop arranged as an 8×8 grid. Dishes interfere if they share a row, column, or diagonal. The company wants to use a **heuristic strategy** to find *one* valid placement quickly, without exhaustively exploring all possibilities.

Heuristic: $h(\text{state}, \text{col})$ = number of positions in future rows that would become unsafe if a dish is placed at `col` next. Always expand the column with the **minimum conflicts**.

Strategy: Greedy Best-First Search using a min-heap (priority queue) ordered by $h(n)$.

Best-First hint: Import `import heapq`. Push tuples `(h_value, tiebreak_counter, state)` onto the heap. The tiebreak counter prevents comparison errors when two states have equal h values. This search finds **one** solution, not all 92.

Tasks:

1. Define the heuristic function `count_conflicts(state, col)` that counts how many future-row positions are blocked by placing a queen at `col`. **[5 Marks]**
2. Write `best_first_queens()` using `heapq` and return the first solution found along with the number of nodes explored. **[7 Marks]**
3. Run your program and record: nodes explored, solution found. **[3 Marks]**
4. **Analysis:** Best-First Search is fast but *not guaranteed to be optimal* and may miss some solutions. Explain with an example why a greedy heuristic can lead the search into a dead end and how the heap handles this. **[5 Marks]**

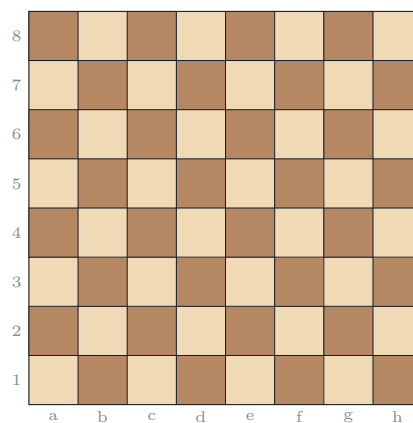
(a) **Heuristic function** `count_conflicts(state, col)` — **pseudocode then Python:**

(b) Python implementation of `best_first_queens()`:

(c) Program output:

Nodes explored: _____ Solution found (1-indexed): _____

Draw the solution on the board:



(d) Analysis — greedy heuristic limitations:

Scenario 4: Radio Telescope Array Positioning

A radio astronomy institute must position **8 telescope arrays** on a grid landscape, one per row, so no two share a column or diagonal. The institute wants the **most informed and efficient search** — fewest states expanded, with a principled cost function.

Cost function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ = number of queens already placed (search depth), and $h(n)$ = number of future rows that currently have **zero safe columns** available (admissible heuristic: rows completely blocked by the current placement).

Strategy: A* Search using a min-heap ordered by $f(n)$.

A* hint: Push tuples (`f`, `g`, `counter`, `state`) onto the heap. Maintain a **closed** set to avoid re-expanding already visited states. The heuristic is **admissible** because a blocked row guarantees at least one additional failure in the remaining search — it never overestimates the cost.

Tasks:

1. Write the admissible heuristic `heuristic(state)` that counts future rows with zero safe columns. **[4 Marks]**
2. Write `astar_queens()` using `heapq` with a closed set. Print the first 10 state expansions showing (g, h, f) values. **[8 Marks]**
3. Run your program and record: total nodes expanded, solution found, and the expansion table. **[4 Marks]**
4. Fill in the comparison table below. **[4 Marks]**
5. **Analysis:** Why is A* considered “optimal” while Best-First is not? What property must $h(n)$ satisfy for A* to be guaranteed optimal? Is the “blocked rows” heuristic admissible — justify your answer. **[5 Marks]**

(a) Heuristic function `heuristic(state)`:

(b) Python implementation of `astar_queens()`:

(c) A* expansion trace (fill in from your program output):

Step	State (column list)	g	h	f
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Total nodes expanded: _____ Solution (1-indexed): _____

(d) **Final comparison table — fill in from your four programs:**

Property	DFS	BFS	Best-First	A*
Data structure used				
Nodes expanded				
Finds all 92 solutions?				
Memory usage				
Uses heuristic?				
Optimal?				
Best use case				

(e) Analysis — admissibility of the heuristic and why A* is optimal: